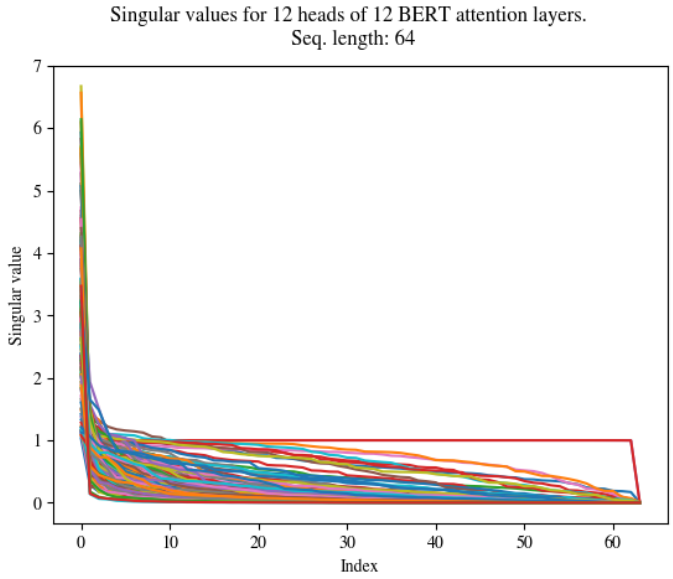
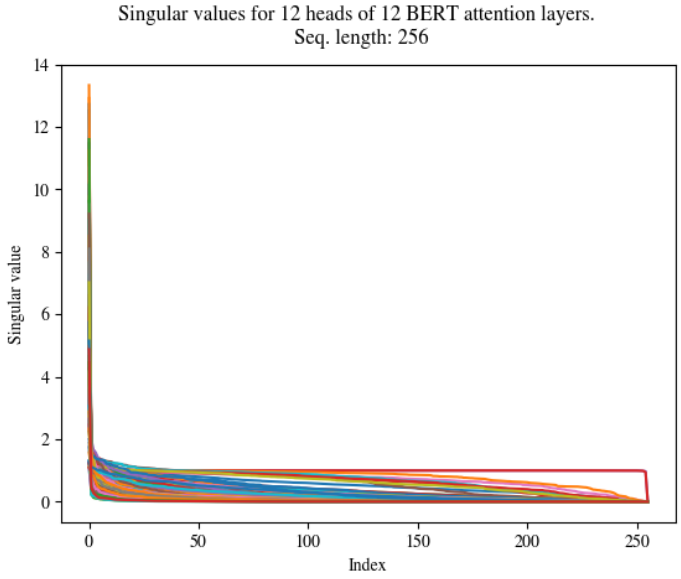


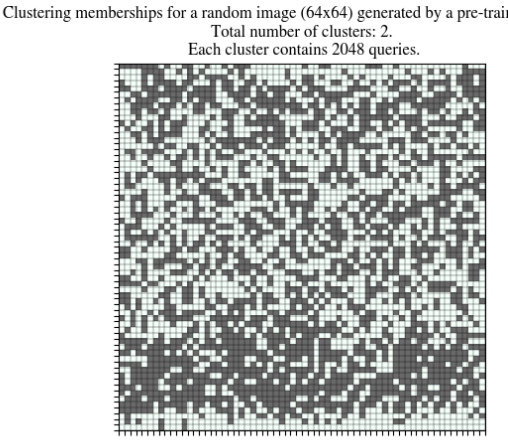
(a)



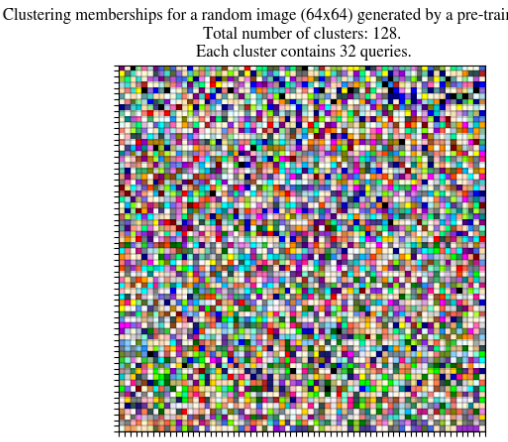
(b)

Figure 6: Decay of singular values for a pre-trained BERT for sequence lengths: (a) 64, (b) 128. We show decay of singular values 144 heads (12 heads for each one of the 12 layers). For the majority of heads, singular values decay exponentially. We also see that the heads that do not demonstrate exponential decay in the singular values maintain this property for both inputs (e.g. see the red line in both plots). We find that these heads are harder to approximate with SMYRF.

(a) Generated maltese dog from a pre-trained BigGAN.



(b) Visualization of SMYRF cluster assignments for this image (single hash). Total number of clusters: 2.



(c) Visualization of SMYRF cluster assignments for this image (single hash). Total number of clusters: 128.

Figure 7: Visualization of clustering assignments for a generated image by a pre-trained BigGAN.

13 SMYRF Clustering

13.1 Asymmetric Locality Sensitive Hashing (ALSH)

SMYRF clusters depend on the hashing indices of asymmetrically transformed queries and keys. As mentioned in the paper, we are looking for functions $F : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}, G : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ such as: $||F(q) - G(k)||_2^2 = D(q \cdot k),\ \forall (q, k)$ where $D : \mathbb{R} \rightarrow \mathbb{R}$ a decreasing function that depends only on the inner product $q \cdot k$. Essentially, functions F, G are applied to queries and keys to convert the problem of Maximum Inner Product Search (MIPS) to Nearest Neighbor Search (NNS). For the latter problem, a lot lot of effective Locality Sensitive Hashing (LSH) functions have been proposed (e.g. [35, 74, 53]). The novel idea of converting MIPS to NNS is called Asymmetric Locality Sensitive Hashing (ALSH) and was first introduced in [32]. Since then, a lot of different asymmetric transformations have been proposed [34, 35, 33]. In this section, we show why previously proposed transformations are not suitable for our problem and how our novel asymmetric transformations, defined in Equation 4, relate to previous work.

We list the asymmetric transformations that have been widely used to convert a MIPS to NNS:

$$\begin{cases} \text{[32]: } F(q_i) = \left[q_i; \frac{1}{2}, ...; \frac{1}{2}\right],\ G(k_i) = \left[Uk_i; ||Uk_i||_2^2; ...; ||Uk_i||_2^{2^m}\right] \\ \text{[33]: } F(q_i) = \left[q_i; 0\right],\ G(k_i) = \left[k_i; \sqrt{M_K^2 - ||k_i||_2^2}\right] \\ \text{[34]: } F(q) = \frac{M_K}{||q||_2} \cdot \left[q; 0\right],\ G(k) = \left[k; \sqrt{M_K^2 - ||k||_2^2}\right] \end{cases}$$

where $M_K = \max_k ||k||_2$ and U a positive constant such as: $||U \cdot k_i||_2^{2^{m+1}} \rightarrow 0,\ \forall k_i \in \mathcal{K}$. The corresponding Euclidean distances of the transformed vectors are given below:

$$\begin{cases} \text{[32]: } ||F(q_i) - G(k_i)||_2^2 = ||q_i||_2^2 + \frac{m}{4} - 2Uq_i \cdot k_i + ||U \cdot k_i||_2^{2^{m+1}} \\ \text{[33]: } ||F(q_i) - G(k_i)||_2^2 = ||q_i||_2^2 + M_K^2 - 2q_i \cdot k_i \\ \text{[34]: } ||F(q_i) - G(k_i)||_2^2 = 2 \cdot M_K^2 - 2\frac{M_K}{||q_i||} \cdot q_i \cdot k \end{cases}$$

In all these transformations the Euclidean distance of the transformed vectors, i.e. $||F(q_i) - G(k_i)||_2$ decreases linearly with the inner product $q_i \cdot k_i$. However, an extra term, $p(||q_i||)$, appears. Indeed, these transformations were proposed for the case of a single query (e.g. a user) and multiple keys (e.g. movies) and for such applications $||q_i||$ is considered constant. On the contrary, for our setting, the transformations of [32, 34, 33] cannot be applied since $||q||_2$ is no longer a constant. To illustrate this better, consider the case where $q_1, q_2 \in \mathcal{Q}$ with $q_1 \neq q_2$ and $k \in \mathcal{K}$ a key such as: $q_1 \cdot k = q_2 \cdot k$. Since we are looking for big inner products, we expect to have transformations $F, Q : ||F(q_1) - G(k)||_2 = ||F(q_2) - G(k)||_2$. For [32, 33], if $||q_1||_2 < ||q_2||_2$ then $||F(q_1) - G(k)||_2 < ||F(q_2) - G(k)||_2$ and for [34]: $||F(q_1) - G(k)||_2 > ||F(q_2) - G(k)||_2$. Thus, all [32, 34, 33] do not satisfy our desired property, i.e. $||F(q_1) - G(k)||_2 = ||F(q_2) - G(k)||_2$. To solve this problem, we propose (see main paper) the novel asymmetric functions:

$$F(q_i) = \left[q_i; 0; \sqrt{M_Q^2 + M_K^2 - ||q_i||_2^2}\right], \qquad G(k_i) = \left[k_i; \sqrt{M_Q^2 + M_K^2 - ||k_i||_2^2}; 0\right] \tag{11}$$

where we use the constants $M_Q = \max_{q_i} ||q_i||_2, \quad M_K = \max_{k_i} ||k_i||_2$, or any other upper bound on the norms. With this transformation, all queries and keys are mapped to a $(d+2)$-dimensional ball with radius $\sqrt{M_Q^2 + M_K^2}$ and the distance of the transformed vectors decreases linearly with the inner product of the original vectors:

$$||F(q_i) - G(k_i)||_2^2 = 2 \cdot \left(M_Q^2 + M_K^2 - q_i \cdot k_i\right). \tag{12}$$

Note that the Euclidean distance of the transformed vectors depends only on the inner product of the original vectors and not on individual norms $||q_i||_2$ as in previous work.

13.2 Adaptive Clustering

The next step, after the asymmetric transformations, is to map the transformed queries $F(q)$ and keys $G(k)$ to real numbers, so that if $||F(q) - G(k)||_2$ is small, then $|h(F(q)) - h(G(k))|$ is also small with high probability, where $h : \mathbb{R}^{d'} \to \mathbb{R}$ is the mapping function. After mapping, we sort independently queries and keys based on their hash and we split them into groups of equal size. There are numerous hashing functions [54, 35, 74, 75] $h : \mathbb{R}^{d'} \to \mathbb{R}$ that belong to the LSH family that we can leverage to achieve that. One of the most widely adopted hash functions for locality sensitive hashing is E2LSH [35]:

$$h_{\text{E2LSH}}(u) = \left\lfloor \frac{(u \cdot a) + b}{r} \right\rfloor \tag{13}$$

where $a = (a_1, ..., a'_d) \in \mathbb{R}^{d'}$ with $a_i \in \mathcal{N}(0, 1)$ and $b \in \mathcal{U}(0, r)$ and r is a scalar parameter which controls LSH sensitivity. Since we re-group vectors by sorting on their LSH index, the floor operator and the division with r are not needed. Our simplified hashing function is defined as:

$$h_{\text{ours}}(u) = (u \cdot a) + b \tag{14}$$

We roughly removed a division by a constant. Thus, this simplified hashing function preserves the locality-sensitive properties of E2LSH [35]. Namely, if $||u_1 - v_1||_2 \leq ||u_2 - v_2||_2$ then with high probability: $|h(u_1) - h(v_1)| \leq |h(u_2) - h(v_2)|,\ \forall u_1, u_2, v_1, v_2 \in \mathbb{R}^{d'}$.

13.3 Merging hashing rounds

In our experiments, we run multiple hashing rounds each time, similarly to [18]. Each time we run LSH, we end up with a (possibly) different clustering assignment and thus (possibly) different attention output. Specifically, we repeat the process H times (where H is usually a small constant, e.g. 8) to reduce the probability that we miss big inner products. In this section, we explain how we merge the partial attention outputs (made from different hashing rounds) into a single attention output.

Without loss of generality, we will present the merging algorithm for a single query q. At each clustering round h we get (from the adaptive clustering) a set of key vectors $\mathcal{K}_{h_q} \subseteq \mathcal{K}$. The corresponding attention output is:

$$o_q^h = \sum_{k \in \mathcal{K}_{h_q}} w_k v_k, \qquad w_k = \frac{e^{q \cdot k}}{\sum_{k' \in \mathcal{K}_{h_q}} e^{q \cdot k'}}$$

We merge the attention outputs of the different rounds with a weighted sum. The weight, a_h, for each round h, is the fraction of the softmax mass that was acquired in this round to the total mass acquired by all rounds. Formally the attention output o'_q for query q is computed as:

$$o'_q = \sum_{h=1}^{H} a_h \cdot \sum_{k \in \mathcal{K}_{h_q}} w_k v_k, \qquad w_k = \frac{e^{q \cdot k}}{\sum_{k' \in \mathcal{K}_{h_q}} e^{q \cdot k'}}, \qquad a_h = \frac{\sum_{k' \in \mathcal{K}_{h_q}} e^{q \cdot k'}}{\sum_{n=1}^{H} \sum_{k' \in \mathcal{K}_{n_q}} e^{q \cdot k'}} \tag{15}$$

To explain this merging scheme, we will show that under certain assumptions, this merging scheme can lead to exact approximation of the real attention output. We start by listing these assumptions.

Assumption 1 (Sparsity of weights)**.** *For any given query $q \in \mathcal{Q}$, the key set $\mathcal{K}$ has at most T and at least one vectors $k_i \in \mathcal{K}_q$ such as:*

$$k_i \in \mathcal{K}_q,\ k_j \notin \mathcal{K}_q \Rightarrow \frac{e^{q \cdot k_j}}{e^{q \cdot k_i}} = 0$$

From Assumption 1, it follows that at most T and at least one key vector k_i gets a non-zero score, $w_i \neq 0$, after softmax.

Assumption 2 (Fairness of LSH clustering)**.** *For any given query $q \in \mathcal{Q}$ and two keys $k_1, k_2 \in \mathcal{K}$, if $w_{k_1} \neq 0 \ \wedge w_{k_2} \neq 0$, then $\sum_{n=1}^{H} \sum_{k_1 \in \mathcal{K}_{n_q}} 1 = \sum_{n=1}^{H} \sum_{k_2 \in \mathcal{K}_{n_q}} 1$ where H denotes the hashing rounds and $\mathcal{K}_{n_q}$ denotes the chosen key set for query q at hash round n.*